```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report

df = pd.read_csv("heart disease dataset.csv")

df.head()

X = df.drop(columns=["target"], axis=1)
X

y = df["target"]
y

from sklearn.preprocessing import StandardScaler
from sklearn.compose import ColumnTransformer

numerical_transfomer = StandardScaler()
## we can use onehot encoder if needed

preprocessor = ColumnTransformer ([
("StandardScaler", numerical_transfomer, X.columns)
])

from sklearn.model_selection import train_test_split

X_train, X_test, Y_train, Y_test = train_test_split(X, y, test_size=0.2, train_size=0.8, random_state=42)


X_train = preprocessor.fit_transform(X_train)
X_test = preprocessor.transform(X_test)

knn = KNeighborsClassifier(n_neighbors=5)

knn.fit(X_train, Y_train)


y_pred = knn.predict(X_test)

accuracy = accuracy_score(Y_test, y_pred)
print("Accuracy:", accuracy)

confusion_matrix(Y_test, y_pred)

print(classification_report(Y_test, y_pred))
```

```
k_values = range(1, 22)
accuracies = []

for k in k_values:
knn = KNeighborsClassifier(n_neighbors=k)
knn.fit(X_train, Y_train)
y_pred = knn.predict(X_test)
acc = accuracy_score(Y_test, y_pred)
accuracies.append(acc)


best_k = k_values[np.argmax(accuracies)]
best_accuracy = max(accuracies)

plt.figure(figsize=(8, 5))
plt.plot(k_values, accuracies, marker='o')
plt.axvline(best_k, linestyle='--', label=f'Best K = {best_k}')

# show accuracy values on the plot
for k, acc in zip(k_values, accuracies):
plt.text(k, acc + 0.002, f"{acc:.2f}", ha='center', fontsize=9)

plt.xlabel("Number of Neighbors (K)")
plt.ylabel("Accuracy")
plt.title("KNN Accuracy for Different Values of K")
plt.xticks(k_values)
plt.legend()
plt.grid(True)
plt.show()

print("Best K:", best_k)
print("Best Accuracy:", best_accuracy)

knn_best = KNeighborsClassifier(n_neighbors=best_k)
knn_best.fit(X_train, Y_train)

y_pred_best = knn_best.predict(X_test)

print("Final Model Accuracy:", accuracy_score(Y_test, y_pred_best))
print("\nConfusion Matrix:\n", confusion_matrix(Y_test, y_pred_best))
print("\nClassification Report:\n", classification_report(Y_test, y_pred_best))

pred_df = pd.DataFrame({"Actual":Y_test, "Predicted":y_pred_best})
pred_df.to_csv("heart_disease_actual_vs_predicted_knn.csv", index=False)
pred_df

# 1. Provide the new patient data (numerical values matching the dataset features)
# Features order: [age, sex, cp, trestbps, chol, fbs, restecg, thalach, exang, oldpeak, slope, ca, thal]
new_patient = [56, 1, 0, 132, 184, 0, 0, 105, 1, 2.1, 1, 1, 1]

# 2. Get the feature column names (excluding the target) from the original dataframe
```

```
# This ensures our new prediction DataFrame has the exact same structure as the training data
X_df = df.drop("target", axis=1)

# 3. Initialize a new DataFrame for the patient with the correct feature names
new_patient_df = pd.DataFrame(
[new_patient],
columns=X_df.columns
)

# 4. Transform/Scale the new data using the pre-fitted preprocessor
# We use .transform() (NOT fit_transform) to apply the exact same scaling used during training
new_patient_scaled = preprocessor.transform(new_patient_df)

# 5. Use the trained KNN model to predict the class
prediction = knn_best.predict(new_patient_scaled)

# 6. Check the prediction result and print a human-readable message
# Class 1 = Heart Disease detected, Class 0 = No Heart Disease
if prediction[0] == 1:
print("Prediction: Heart Disease")
else:
print("Prediction: No Heart Disease")

import joblib
joblib.dump(knn_best, "knn_model.pkl")
joblib.dump(preprocessor, "preprocessor.pkl")
```